

ATLAS

Projet fil rouge React

16 paliers, du premier composant aux hooks avancés

Cahier de l'élève

Formation React — Modules 1 & 2

1. Le projet

Tu vas construire **Atlas**, un explorateur de pays du monde. Une seule application, qui grossit palier après palier. À la fin, elle permettra de :

- parcourir les pays du monde sous forme de cartes avec leur drapeau,
- rechercher un pays par son nom,
- filtrer par région et trier par population ou superficie,
- mettre des pays en favoris, conservés après un rechargement de page,
- ouvrir le détail d'un pays dans une fenêtre modale,
- charger la suite automatiquement au scroll,
- basculer entre thème clair et thème sombre,
- jouer à un mini-quiz chronométré « quel pays a ce drapeau ? ».

Chaque palier ajoute **une seule** notion. À la fin de chaque palier, ton application doit toujours fonctionner. Si elle est cassée, ne passe pas au suivant.

Ce que couvre le projet

Palier	Titre	Notions
0	Mise en place	Bundler, Vite, architecture d'un projet
1	Premier composant	JSX, composant, import / export
2	La carte pays	Props, déstructuration
3	La grille	Listes, <code>.map()</code> , <code>key</code> , images
4	Le favori	<code>useState</code> , événements, callback props
5	Les états d'affichage	<code>if</code> , ternaire, <code>&&</code> , classe conditionnelle
6	La recherche	Input contrôlé et non contrôlé
7	Les vraies données	<code>useEffect</code> , appel API, les trois états
8	Le grand nettoyage	Fragments, règle des hooks, hook personnalisé
9	Les favoris solides	<code>useReducer</code>
10	La persistance	Hook personnalisé, <code>localStorage</code>
11	Le thème	Context API, Provider, hook d'accès
12	Le scroll infini	<code>useRef</code> , <code>IntersectionObserver</code> , événement global
13	Le mode Quiz	<code>setInterval</code> , <code>cleanup</code> , tableau de refs
14	Les composants boîte	<code>props.children</code> , composition
15	La performance	<code>useMemo</code> , <code>React.memo</code> , <code>useCallback</code> , mesure
16	Le style	CSS Modules, Tailwind

2. Comment travailler

Les quatre règles

1. **Fais tourner l'application après chaque ajout.** Pas après trente lignes. Si tu écris beaucoup avant de tester, tu ne sauras pas quelle ligne est fausse.
2. **Lis les messages d'erreur en entier.** Ils disent presque toujours ce qui ne va pas et où. C'est une compétence à part entière.
3. **Cherche cinq minutes, puis ouvre l'aide.** Bloquer trente minutes n'apprend rien. Trouver seul après cinq minutes de recherche, si.
4. **Garde la console ouverte en permanence.** Les avertissements React sont précieux, ne les ignore pas.

Les aides

Chaque palier propose trois niveaux d'aide. Ils sont volontairement progressifs : descends d'un cran seulement quand tu as vraiment essayé.

Niveau	Ce que ça te donne
Piste	La direction à prendre, sans technique. « Regarde du côté de... »
Indice	Le nom de la fonction ou du hook, la syntaxe utile.
Squelette	La structure du code, avec les trous à remplir toi-même.

Il n'y a pas de corrigé dans ce document. C'est volontaire. Un corrigé se lit en trente secondes et s'oublie en une heure ; un problème résolu reste. Ton formateur a les solutions : demande-lui après avoir cherché.

UNE HABITUDE À PRENDRE DÈS MAINTENANT

Fais un commit Git à la fin de chaque palier, avec un message du type `palier 4 : favoris`. Tu auras un historique lisible de ta progression, et surtout un point de retour propre si tu casses tout au palier suivant.

3. L'API

Les données viennent de **REST Countries v5**, sur `api.restcountries.com`. Cette version demande une **clé d'API** : on ne peut plus appeler l'API sans s'identifier, contrairement aux versions précédentes.

Obtenir ta clé

1. Crée un compte sur `restcountries.com/sign-up`. Le palier gratuit couvre 500 requêtes par mois, sans carte bancaire.
2. Récupère ta clé dans la page « API Keys ».
3. Il existe aussi une clé de démonstration, `rc_live_demo`, sans inscription — mais elle ne renvoie **qu'un échantillon**, pas la liste complète. Elle sert à vérifier que l'API répond, pas à faire tourner le projet.

La forme d'une requête

```
const response = await fetch(
  'https://api.restcountries.com/countries/v5?q=canada',
  { headers: { 'Authorization': 'Bearer TA_CLE' } }
);
const result = await response.json();
```

Trois choses à retenir : l'URL est sur `api.restcountries.com`, la clé passe dans un **en-tête** `Authorization`, et la réponse est un objet JSON dont les données se trouvent dans une propriété — pas directement un tableau.

500 REQUÊTES PAR MOIS, C'EST PEU

Avec le rechargement à chaud, **chaque sauvegarde de fichier peut relancer une requête**. Une après-midi de développement peut consommer ton quota entier.

La règle du projet : dès que ton appel API fonctionne (palier 7), tu mets en cache la réponse dans le `localStorage` et tu ne rappelles l'API que si le cache est vide. C'est expliqué dans le palier 7. Ce n'est pas une contrainte artificielle : gérer un quota fait partie du métier.

UNE CLÉ DANS UNE APPLICATION FRONT, ÇA SE VOIT

Tout ce qui est dans ton code JavaScript est lisible par n'importe qui ouvrant les outils de développement. Une clé d'API mise dans une application React **n'est pas secrète**, même stockée dans un fichier `.env` : Vite l'inclut simplement dans le fichier final.

Pour ce projet d'apprentissage, c'est acceptable. En production, une clé sensible doit rester sur un serveur qui fait l'appel à ta place. Retiens cette distinction, on te la demandera en entretien.

Découvrir les données

Avant d'écrire la moindre ligne de React, exécute une requête et regarde la réponse. Tu dois savoir répondre à ces questions :

- Où se trouve le tableau des pays dans la réponse ?
- Comment récupère-t-on le nom courant d'un pays ? (indice : ce n'est pas `name`)
- Où est l'URL du drapeau ? La population ? La capitale ?
- Quel champ peut servir d'identifiant unique ?
- Y a-t-il des champs parfois absents ? (regarde l'Antarctique)

Note tes réponses sur papier. Tu t'en serviras au palier 7. **Lire une réponse d'API est une compétence du métier** : aucun document ne te donnera jamais la structure exacte à ta place.

SI L'API NE RÉPOND PAS

Une API publique peut tomber, changer, ou refuser ton quota. Ton formateur a un fichier `countries.json` de secours à importer dans `src/data/`. Le projet fonctionne à l'identique : tu sautes simplement l'appel réseau du palier 7 et tu y reviens plus tard.

Mise en place

Bundler · Vite · architecture d'un projet React

≈ 20 min

OBJECTIF

Avoir un projet qui tourne, et comprendre à quoi sert chaque fichier.

Ce que tu dois faire

A. Créer le projet :

```
npm create vite@latest atlas  
# Framework : React  
# Variant : JavaScript  
  
cd atlas  
npm install  
npm run dev
```

B. Ouvrir et lire, dans cet ordre : `index.html` , puis `src/main.jsx` , puis `src/App.jsx` . Pour chacun, écris en une phrase ce qu'il fait.

C. Faire l'observation suivante : ouvre `index.html` dans ton éditeur — il est quasiment vide. Puis inspecte la page dans le navigateur — le DOM est rempli. *Explique pourquoi.*

D. Faire le ménage : vider `App.css` et `index.css` (garder les fichiers), et réduire `App.jsx` à un simple titre « Atlas ».

E. Créer l'arborescence de travail :

```
src/  
  components/  
  hooks/  
  contexts/  
  data/
```

CRITÈRES DE RÉUSSITE

- ☐ `npm run dev` démarre et la page affiche « Atlas »
- ☐ Modifier le texte met à jour le navigateur sans rechargement manuel
- ☐ Je sais expliquer le rôle de `index.html` , `main.jsx` et `App.jsx`
- ☐ Les dossiers de travail sont créés
- ☐ Le projet est versionné (`git init`) et un premier commit est fait

AIDE

Piste

Pour la question C : cherche dans `index.html` une balise vide qui porte un identifiant. Puis cherche cet identifiant dans `main.jsx`.

Indice

Le bundler (ici Vite) traduit le JSX que le navigateur ne comprend pas, regroupe les fichiers, et sert le tout avec rechargement à chaud. Sans lui, ton code ne s'exécuterait pas dans un navigateur.

Squelette

```
function App() {  
  return <h1>Atlas</h1>;  
}  
  
export default App;
```

Premier composant

JSX · composant · import / export

≈ 20 min

OBJECTIF

Écrire un composant, l'exporter, l'importer, l'utiliser.

Ce que tu dois faire

Créer `src/components/Header.jsx`, un en-tête contenant :

- le titre « Atlas »,
- le sous-titre « Explorateur de pays »,
- une ligne « X pays à découvrir », où X vient d'une **variable JavaScript** (mets 250 en dur pour l'instant).

Puis l'utiliser dans `App.jsx` à la place du titre écrit en dur.

Contraintes

- le composant retourne **un seul élément parent**,
- la classe CSS s'écrit avec le bon attribut (ce n'est pas `class`),
- le nom du composant respecte la convention de nommage React.

CRITÈRES DE RÉUSSITE

- ☐ « Atlas » s'affiche via `<Header />` et non plus en dur dans `App`
- ☐ Il y a un `export default` dans `Header.jsx` et un `import` dans `App.jsx`
- ☐ Une valeur JavaScript est affichée entre accolades
- ☐ Aucune erreur ni avertissement dans la console

AIDE

Piste

Un composant React est une fonction ordinaire qui retourne du JSX. Si rien ne s'affiche alors que ton code semble correct, regarde très attentivement la **première lettre** du nom de ta fonction.

Indice

Trois points de syntaxe : `className` et non `class` (mot réservé en JavaScript) ; les accolades `{ }` pour insérer une *expression* JavaScript dans le JSX ; un seul élément racine dans le `return`.

Squelette

```
// src/components/Header.jsx
function Header() {
  const nombreDePays = /* ... */;

  return (
    <header className="header">
      {/* le titre, le sous-titre */}
      {/* une ligne qui affiche nombreDePays */}
    </header>
  );
}
```

```
    </header>
  );
}

export default /* ... */;
```


La carte pays

Props · déstructuration · lecture seule

≈ 25 min

OBJECTIF

Faire un composant réutilisable, configuré de l'extérieur.

Ce que tu dois faire

A. Créer `src/components/CountryCard.jsx`, qui affiche le nom d'un pays, sa capitale, sa population et sa région.

B. Dans `App.jsx`, afficher **trois cartes différentes** — France, Japon, Brésil — avec des valeurs écrites en dur, en utilisant le même composant trois fois.

C. Formater la population pour qu'elle s'affiche `67 391 582` et non `67391582`.

Contraintes

- utiliser la **déstructuration** dans la signature du composant,
- faire attention au **type** des valeurs passées : la population est un nombre, pas du texte.

CRITÈRES DE RÉUSSITE

- ☐ Trois cartes différentes s'affichent avec un seul composant
- ☐ La population est formatée avec des séparateurs de milliers
- ☐ Je sais dire pourquoi certaines props ont des guillemets et d'autres des accolades
- ☐ Je sais expliquer pourquoi un composant ne doit jamais modifier ses props

AIDE

Piste

Les props arrivent toutes dans un seul objet, premier paramètre de la fonction. Tu peux soit le recevoir entier, soit en extraire directement les champs qui t'intéressent.

Pour le formatage : la solution est une méthode JavaScript native des nombres. Ce n'est pas du React. Cherche « formater un nombre avec des espaces JavaScript ».

Indice

`population.toLocaleString("fr-FR")`. Et la différence à comprendre : `age="26"` passe la chaîne "26", `age={26}` passe le nombre 26. Une méthode de nombre appelée sur une chaîne échouera.

Squelette

```
function CountryCard({ /* les champs attendus */ }) {  
  return (  
    <article className="card">  
      { /* ... */ }  
    </article>  
  );  
}
```

```
// Dans App.jsx  
<CountryCard name="France" capital="Paris" population={/* ... */} ... />
```

La grille

Listes · `.map()` · `key` · images

≈ 30 min

OBJECTIF

Générer les cartes depuis un tableau de données au lieu de les écrire une par une.

Ce que tu dois faire

A. Créer `src/data/countries.js` qui exporte un tableau de 8 pays. Chaque pays est un objet avec `id`, `name`, `capital`, `population`, `region` et `flag` (l'URL du drapeau).

Pour les drapeaux, tu peux utiliser : `https://flagcdn.com/w320/fr.png` (code pays en minuscules).

B. Dans `App.jsx`, générer les cartes automatiquement à partir de ce tableau.

C. Ajouter le drapeau dans `CountryCard`, avec un attribut `alt` descriptif **construit à partir du nom du pays**.

L'expérience à mener

Fais-la vraiment, elle vaut mieux qu'une explication :

1. Génère la liste **sans** attribut `key`. Regarde la console.
2. Mets `key={index}`. L'avertissement disparaît.
3. Ajoute temporairement un `<input />` vide dans chaque carte. Tape du texte dans le deuxième champ, puis supprime le premier pays du tableau. **Observe où va le texte.**
4. Remplace par `key={country.id}` et refais l'expérience.

Écris en une phrase ce que tu as observé, et pourquoi.

CRITÈRES DE RÉUSSITE

- ☐ 8 cartes s'affichent, générées automatiquement
- ☐ Aucun avertissement `key` dans la console
- ☐ Chaque image a un `alt` descriptif et dynamique
- ☐ Je sais expliquer à quoi sert vraiment la `key`, au-delà de « React rôle sinon »

AIDE

Piste

`.map()` n'est pas une fonctionnalité React : c'est une méthode standard des tableaux JavaScript. Elle transforme un tableau de données en un tableau d'autre chose — ici, de composants.

Si rien ne s'affiche : vérifie si ta fonction fléchée *retourne* quelque chose. Avec des accolades, il faut un `return` explicite ; avec des parenthèses, il est implicite.

Indice

La `key` se place sur l'élément le plus **extérieur** retourné par le `.map()`, pas à l'intérieur. Pour l'`alt` dynamique, pense aux *template literals* : `{`Drapeau : ${name}`}`.

Squelette

```
{countries.map(country => (  
  <CountryCard  
    key={/* ... */}  
    /* les props, une par une */  
  />  
))}
```

POUR ALLER PLUS LOIN

Cherche ce que fait la syntaxe `{...country}` passée à un composant. Compare avec ta version où tu écris chaque prop. Laquelle préfères-tu, et pourquoi ? Il n'y a pas de bonne réponse, mais il faut savoir défendre la sienne.

Le favori

useState · événements · callback props

≈ 40 min

OBJECTIF

Rendre l'interface interactive, et découvrir où doit vivre une donnée partagée.

Ce que tu dois faire

A — L'expérience à mener d'abord. Dans `CountryCard`, ajoute un bouton étoile et une **variable normale** `let estFavori = false`. Au clic, inverse la variable et affiche-la dans la console.

Clique plusieurs fois. La console change. L'écran, non. *Écris pourquoi avant de continuer.*

B. Remplace la variable par un `useState`, toujours dans `CountryCard`. L'étoile bascule maintenant entre ☆ et ★.

C — Le nouveau problème. On veut afficher « X favoris » dans le `Header`. Essaie. Tu vas constater que c'est impossible en l'état.

Pourquoi ? Réponds avant de lire la suite.

D. Déplace le state des favoris dans `App` : `App` détient un tableau des identifiants favoris, le `Header` reçoit le nombre, et chaque carte reçoit deux choses — si elle est en favori, et un moyen de le signaler.

E. Vérifie que le compteur du `Header` se met bien à jour.

CRITÈRES DE RÉUSSITE

- ☐ Cliquer sur l'étoile la fait basculer
- ☐ Le compteur du `Header` se met à jour en direct
- ☐ `CountryCard` n'a plus aucun `useState` pour le favori
- ☐ Je sais expliquer pourquoi une variable normale ne suffit pas
- ☐ Je sais dire où doit vivre une donnée dont deux composants ont besoin

AIDE

Piste

Pour A : souviens-toi de ce qu'est un composant. Une fonction que React appelle pour savoir quoi afficher. Si personne ne rappelle la fonction, l'écran ne peut pas changer — et rien ne prévient React que ta variable a bougé.

Pour C et D : quand deux composants ont besoin de la même information, elle ne peut pas vivre dans l'un des deux. Cherche l'ancêtre commun.

Indice

Le parent passe une **fonction** en prop à l'enfant ; l'enfant l'appelle quand il se passe quelque chose. Les données descendent, les événements remontent.

Attention à deux choses : `onClick={maFonction}` et non `onClick={maFonction()}` ; et `.push()` sur un tableau de state ne déclenche **aucun** re-render — cherche pourquoi, c'est une notion importante.

Squelette

```
// App.jsx
const [favoris, setFavoris] = useState([]); // tableau d'id

function basculerFavori(id) {
  setFavoris(actuels => {
    // si l'id est déjà dedans → retourner un tableau SANS lui
    // sinon → retourner un NOUVEAU tableau AVEC lui
  });
}

<CountryCard
  estFavori={/* ... */}
  onToggleFavorite={/* ... */}
/>
```

LE VOCABULAIRE

Ce que tu viens de faire en D s'appelle **faire remonter l'état** (*lifting state up*). C'est un des gestes les plus fréquents en React. Retiens le terme, il reviendra.

Les états d'affichage

if · ternaire · && · classe conditionnelle

≈ 30 min

OBJECTIF

Afficher des choses différentes selon les données, avec les trois techniques du cours.

Ce que tu dois faire

Cinq ajouts, chacun avec une technique imposée.

A — Avec un `if` avant le `return`. Si la capitale est absente (`null`), afficher une carte simplifiée « Données incomplètes » sans la ligne Capitale. Ajoute un pays sans capitale à ton fichier de données pour tester.

B — Avec un `ternaire`. Un badge « Grand pays » si la population dépasse 50 millions, « Petit pays » sinon.

C — Avec un `&&`. Afficher « Dans vos favoris » sous la carte, uniquement si le pays est en favori.

D — Une classe conditionnelle. Ajouter la classe `card--favorite` quand le pays est en favori, et lui donner une bordure dorée en CSS.

E — L'expérience à mener. Ajoute une prop `nombreDeLangues`. Affiche « X langue(s) » seulement s'il y en a, avec un `&&`. Puis passe volontairement `nombreDeLangues={0}` sur une carte.

Regarde l'écran. Que s'affiche-t-il ? Pourquoi ? Corrige.

CRITÈRES DE RÉUSSITE

- ☐ Les quatre techniques sont utilisées au moins une fois chacune
- ☐ Un pays sans capitale s'affiche sans planter
- ☐ Un pays à 0 langue n'affiche pas de 0 parasite
- ☐ La carte favorite est visuellement distincte
- ☐ Je sais dire quelle technique choisir dans quelle situation

AIDE

Piste

Le critère de choix : `if` quand les deux rendus sont vraiment différents ; ternaire quand il y a toujours quelque chose à afficher, mais deux versions ; `&&` quand c'est « ça, ou rien ».

Pour E : réfléchis à ce que *retourne* l'opérateur `&&` quand la valeur de gauche est fausse. Puis demande-toi ce que React affiche de cette valeur.

Indice

React ignore `false`, `null` et `undefined`, mais **affiche** les nombres — y compris `0`. Mets toujours un vrai booléen à gauche d'un `&&`.

Pour D : une classe CSS n'est qu'une chaîne de caractères. Donc tu peux la calculer, avec un ternaire dans un *template literal*.

Squelette

```
// A
if (!capital) {
  return ( /* la carte simplifiée */ );
}

// D
const classes = `card ${/* ... */}`;

// E
{ /* la condition corrigée */ && <p>...</p> }
```

POUR ALLER PLUS LOIN

Ajoute un troisième palier au badge : « Très grand » au-dessus de 100 millions. Fais-le d'abord avec des ternaires imbriqués, relis le résultat, puis cherche une écriture plus lisible. Cette réécriture est le vrai exercice.

La recherche et les filtres

Input contrôlé · input non contrôlé · formulaire

≈ 45 min

OBJECTIF

Filtrer la liste en direct, et comprendre les deux façons de gérer un champ de saisie.

Ce que tu dois faire

- A.** Un champ de recherche dans `App`. À chaque frappe, la grille ne montre que les pays dont le nom contient le texte saisi. La recherche doit être **insensible à la casse**.
- B.** Un `<select>` de filtre par région, avec une option « Toutes ». Les deux filtres doivent se **combiner**.
- C.** Un bouton « Réinitialiser » qui vide les deux filtres.
- D.** Gérer l'état vide : si aucun pays ne correspond, afficher un message à la place de la grille.
- E — La comparaison.** Créer un composant `AllerAuPays` : un petit formulaire **non contrôlé**, qui lit sa valeur uniquement à la soumission (avec une `ref`) et affiche une alerte avec le pays trouvé.

Compare les deux approches et note en deux lignes ce qui les distingue.

LE PIÈGE DE CE PALIER

Tu vas être tenté de créer un `useState` pour stocker la liste filtrée. **Ne le fais pas.**

Demande-toi : cette liste peut-elle se déduire entièrement de ce que je sais déjà (le texte recherché, la région choisie) ? Si oui, ce n'est pas un état, c'est un **calcul**. Deux états qui décrivent la même chose finissent toujours par se contredire.

CRITÈRES DE RÉUSSITE

- ☐ La recherche filtre à chaque frappe, sans tenir compte de la casse
- ☐ Les deux filtres se combinent correctement
- ☐ L'état « aucun résultat » est géré
- ☐ Il n'y a **aucun** state pour la liste filtrée
- ☐ Le formulaire non contrôlé ne recharge pas la page à la soumission
- ☐ Je sais expliquer qui détient la valeur dans chacune des deux approches

AIDE

Piste

Input contrôlé : la valeur affichée *vient* du state, et chaque frappe met le state à jour. Ces deux moitiés vont toujours ensemble : l'une sans l'autre donne un champ dans lequel on ne peut pas taper.

Pour la liste filtrée : c'est une simple variable, calculée dans le corps du composant, avant le `return`.

Indice

Côté contrôlé : `value` + `onChange`, et `e.target.value` pour lire la saisie. Côté non contrôlé : `useRef`, l'attribut `ref`, et `maRef.current.value` au moment de la soumission.

Sur un `<form>`, `e.preventDefault()` est obligatoire dans le `onSubmit`, sinon le navigateur recharge la page. C'est du HTML natif, pas du React.

Squelette

```
const [recherche, setRecherche] = useState("");
const [region, setRegion] = useState("toutes");

// une VARIABLE, pas un state
const paysFiltres = countries.filter(pays => {
  const correspondNom = /* ... */;
  const correspondRegion = /* ... */;
  return correspondNom && correspondRegion;
});
```

LA RÈGLE À RETENIR DE CE PALIER

Si une valeur peut être calculée à partir d'un état existant, ne la stocke pas dans un état. Calcule-la. C'est probablement la règle qui fait le plus de différence entre du code React sain et du code qui devient ingérable.

Les vraies données

useEffect · appel API · les trois états de l'interface

≈ 60 min — le palier le plus difficile

OBJECTIF

Remplacer le fichier de données en dur par un vrai appel réseau, proprement.

AVANT DE COMMENCER

Ouvre la console de ton navigateur et exécute une requête vers l'API, sans React. Regarde la réponse. Si tu ne sais pas expliquer ce que fait ce code, **arrête-toi et revois les promesses en JavaScript** — quinze minutes maintenant t'en feront gagner deux heures.

```
fetch(url, { headers: { Authorization: 'Bearer TA_CLE' } })
  .then(res => res.json())
  .then(data => console.log(data));
```

Le point à comprendre : `fetch` ne te rend pas les données. Il te rend une **promesse** de données, qui arriveront plus tard.

Ce que tu dois faire — quatre étapes, on teste entre chaque

Étape A — Le minimum. Un state pour les données, un `useEffect` qui se déclenche une seule fois, un `fetch`, et les pays s'affichent. Pas encore de chargement, pas encore d'erreurs.

L'observation obligatoire : ajoute cette ligne tout en haut de ton composant, avant le `return` :

```
console.log("RENDU — nombre de pays :", pays.length);
```

Recharge la page et regarde la console. **Combien de fois ton composant s'affiche-t-il ? Avec quelles valeurs ?** Dessine la chronologie sur papier. C'est l'observation la plus importante de toute la formation.

Étape B — Le chargement. Ajoute un état booléen et un écran « Chargement... ». Pour le voir, passe ton onglet Réseau en « Slow 3G ».

Étape C — Les erreurs. Ajoute un état d'erreur, un `try / catch / finally`, et une vérification du statut de la réponse. **Teste en cassant volontairement l'URL ou la clé.** Une gestion d'erreur jamais testée n'existe pas.

Étape D — Le mapping. L'API ne renvoie pas les données dans la forme dont ton application a besoin. Transforme-les en objets simples, une bonne fois pour toutes, juste après la réception. Sers-toi des notes que tu as prises en section 3.

OBLIGATOIRE : LE CACHE, POUR TON QUOTA

Dès que l'étape D fonctionne, ajoute ceci : **si les données sont déjà dans le `localStorage`, on les utilise et on n'appelle pas l'API.** Sinon, on appelle l'API et on range le résultat.

Sans ça, chaque sauvegarde de fichier consomme une requête sur les 500 de ton quota mensuel. Prévois aussi un bouton « Vider le cache » pour tes tests.

CRITÈRES DE RÉUSSITE

- ☐ Les pays s'affichent depuis l'API
- ☐ « Chargement... » est visible en réseau lent
- ☐ Casser l'URL affiche un message d'erreur, pas un écran blanc
- ☐ L'écran ne reste jamais bloqué sur « Chargement... », même en cas d'erreur
- ☐ La recherche et les filtres du palier 6 fonctionnent toujours
- ☐ Le cache fonctionne : recharger la page ne déclenche pas de nouvelle requête
- ☐ Je sais dire combien de fois mon composant s'affiche pendant un chargement, et pourquoi

AIDE

Piste

Pourquoi un `useEffect` et pas un `fetch` écrit directement dans le composant ? Déroule ce qui se passerait : le composant s'affiche, le fetch part, les données arrivent, le state change, le composant s'affiche... et le fetch repart. Dessine la boucle.

Pour l'initialisation du state des données : réfléchis à ce que vaut ta variable au tout premier rendu, et à ce que ferait `.map()` dessus.

Indice

Cinq points techniques :

- Le tableau de dépendances vide `[]` signifie « une seule fois, au montage ».
- `useEffect` ne peut pas être `async`. Définis une fonction `async` à l'intérieur, puis appelle-la.
- `fetch` ne rejette **pas** sur une erreur 404 ou 500. Vérifie `res.ok` toi-même et lève l'erreur.
- Le `finally` s'exécute dans tous les cas — c'est là que va la fin du chargement.
- Une liste s'initialise à `[]`, jamais à `null`.

Squelette

```
const [pays, setPays] = useState(/* ... */);
const [chargement, setChargement] = useState(/* ... */);
const [erreur, setErreur] = useState(/* ... */);

useEffect(() => {
  async function charger() {
    try {
      // vérifier le cache d'abord
      // sinon : fetch, vérifier res.ok, res.json(), transformer, stocker
    } catch (err) {
      // ...
    } finally {
      // ...
    }
  }
  charger();
}, [/* ... */]);

// après TOUS les hooks :
```

```
if (chargement) return /* ... */;  
if (erreur) return /* ... */;
```

LE SCHÉMA À RETENIR

Une interface qui affiche des données a **trois écrans possibles**, pas un : chargement, succès, erreur. Le débutant ne code que le troisième — et son application montre du blanc pendant deux secondes, puis plante en silence le jour où l'API tombe.

Le grand nettoyage

Fragments · règle des hooks · premier hook personnalisé

≈ 40 min

OBJECTIF

App.jsx fait maintenant plus de cent lignes et mélange tout. On range.

Ce que tu dois faire

A — Les fragments. Repère dans ton code les `<div>` qui n'existent que pour satisfaire la règle du parent unique, et remplace-les. Inspecte le DOM avant et après pour voir la différence.

B — L'expérience sur la règle des hooks. Écris volontairement ce code, puis fais tourner l'application et clique jusqu'à ce que la condition change :

```
const [chargement, setChargement] = useState(true);

if (chargement) {
  const [pays, setPays] = useState([]); // à ne jamais faire
}
```

Note ce qui se passe. Puis cherche à comprendre **pourquoi**. Une piste pour t'aider : React ne connaît pas le nom de tes variables — la déstructuration se fait de ton côté. Comment fait-il alors pour retrouver la bonne valeur d'un rendu à l'autre ?

C — Le hook personnalisé. Extrais toute la logique de chargement dans `src/hooks/useCountries.js`. Ce hook doit retourner les données, l'état de chargement et l'erreur. `App.jsx` doit se réduire à une seule ligne pour tout ça.

D — Découpe les composants. Sors les filtres et la grille dans leurs propres fichiers. Objectif : aucun fichier de plus de 100 lignes.

CRITÈRES DE RÉUSSITE

- ☐ Plus aucune `div` inutile dans le DOM inspecté
- ☐ Je sais expliquer pourquoi un hook ne peut pas être dans un `if`
- ☐ `useCountries` existe, et `App.jsx` a nettement maigri
- ☐ Aucun fichier ne dépasse 100 lignes
- ☐ L'application fonctionne exactement comme avant

AIDE

Piste

Un hook personnalisé n'est pas une nouvelle notion : c'est un simple **copier-coller** de ta logique dans une fonction. Il y a une seule contrainte sur son nom.

Pour la règle des hooks : React tient une liste de valeurs par composant, et il les retrouve par leur **ordre d'appel**. Dessine cette liste sur trois rendus successifs, avec le `if`, et tu verras le problème.

Indice

Le nom doit commencer par `use` : ce n'est pas décoratif, c'est ce qui permet aux outils d'analyse d'y appliquer les règles des hooks.

Un hook retourne des **données**, jamais du JSX. Pour trois valeurs ou plus, retourne un objet plutôt qu'un tableau.

Attention aussi : un `return` anticipé placé *avant* un hook est la même erreur qu'un `if` autour. Les hooks d'abord, les sorties ensuite.

Squelette

```
// src/hooks/useCountries.js
export function useCountries() {
  // exactement le code que tu avais dans App
  return { /* ... */ };
}

// src/App.jsx
const { pays, chargement, erreur } = useCountries();
```

QUESTION À TE POSER

Si deux composants différents appelaient `useCountries()`, partageraient-ils les mêmes données ? Ou chacun aurait-il les siennes ? Réfléchis avant de tester, puis teste. La réponse t'apprend ce qu'un hook partage exactement — et ce qu'il ne partage pas.

Les favoris solides

useReducer

≈ 50 min

OBJECTIF

Rencontrer le problème que `useReducer` résout, puis le résoudre.

Ce que tu dois faire

A — D'abord, avec ce que tu sais. Ajoute ces fonctionnalités en utilisant uniquement `useState` :

- un bouton « Tout vider » pour les favoris,
- un tri : par nom, par population, par superficie,
- un ordre de tri croissant / décroissant, qui s'inverse si on reclique sur le même critère,
- un mode « afficher uniquement mes favoris »,
- un bouton « Réinitialiser » qui remet les filtres à zéro **sans** effacer les favoris.

Écris tout ça. Compte tes `useState`. Compte, dans chaque fonction de gestion, combien de setters tu appelles à la suite. **Note ces deux chiffres.**

B — Puis, avec un reducer. Regroupe tout ce qui concerne l'affichage de la liste — recherche, région, tri, ordre, mode favoris — dans un seul `useReducer`, dans `src/reducers/listeReducer.js`.

C. Compare les deux versions. Où se trouve maintenant la logique ? Que se passe-t-il si tu dois ajouter une sixième fonctionnalité ?

CRITÈRES DE RÉUSSITE

- ☐ Toutes les fonctionnalités marchent
- ☐ Un seul `useReducer` remplace les états d'affichage
- ☐ Le reducer est dans son propre fichier, en dehors du composant
- ☐ Le reducer ne contient aucun appel réseau, aucun timer, aucun aléatoire
- ☐ Je sais dire pourquoi le champ de saisie du formulaire non contrôlé n'a pas besoin d'être dans le reducer
- ☐ Je sais énoncer le signal qui indique qu'il faut passer à un reducer

AIDE

Piste

Le vocabulaire, dans l'ordre : un **état** (un seul objet), une **action** (un objet qui décrit ce qui vient de se passer), `dispatch` (envoyer une action), le **reducer** (la fonction qui reçoit l'état et l'action, et retourne le nouvel état).

Une image : c'est un distributeur automatique. Tu n'ouvres pas la machine pour prendre ce que tu veux ; tu appuies sur B4. Tu dis *ce que tu veux*, la machine décide *comment*.

Indice

Trois points sur lesquels tout le monde se casse les dents la première fois :

- Le reducer doit retourner un **nouvel objet**. Si tu modifies l'ancien et que tu le retournes, React voit la même référence et ne re-render pas.
- Il faut un cas par défaut qui retourne l'état inchangé.
- Le reducer est une fonction **pure** : mêmes entrées, même sortie, aucun effet de bord.

Squelette

```
export const etatInitial = { /* tous les champs d'affichage */ };

export function listeReducer(etat, action) {
  switch (action.type) {
    case "RECHERCHER":
      return { ...etat, /* ... */ };
    case "TRIER":
      // attention : re cliquer sur le même critère inverse l'ordre
      return /* ... */;
    // ...
    default:
      return etat;
  }
}

// dans le composant
const [etat, dispatch] = useReducer(listeReducer, etatInitial);
dispatch({ type: "RECHERCHER", payload: /* ... */ });
```

LE VRAI TEST DE CE PALIER

Dans ton reducer, l'action « tout vider les favoris » doit aussi désactiver le mode « favoris seulement » — sinon l'utilisateur se retrouve devant un écran vide sans comprendre. Écris cette règle. Puis demande-toi : avec des `useState`, à combien d'endroits aurait-il fallu la répéter ?

La persistance

Hook personnalisé · localStorage · initialisation paresseuse

≈ 30 min

OBJECTIF

Les favoris survivent au rechargement de la page.

Ce que tu dois faire

A. Écrire `src/hooks/useLocalStorage.js`. Il doit s'utiliser **exactement comme** `useState` : le composant qui l'appelle ne doit voir aucune différence.

```
const [valeur, setValeur] = useLocalStorage("ma-cle", valeurParDefaut);
```

B. L'utiliser pour les favoris. Deux approches sont possibles, à toi de choisir et de justifier :

- sortir les favoris du reducer et les gérer avec `useLocalStorage` à part,
- ou les garder dans le reducer et ajouter un effet qui sauvegarde à chaque changement.

C. Tester : ajouter trois favoris, appuyer sur F5, vérifier qu'ils sont toujours là.

D. Préfixer ta clé de stockage avec le nom du projet (`atlas:favoris`). Cherche pourquoi c'est une bonne pratique.

CRITÈRES DE RÉUSSITE

- ☐ Les favoris survivent au rechargement
- ☐ `useLocalStorage` s'utilise comme `useState`, sans que le composant sache ce qu'il y a derrière
- ☐ Le hook ne plante pas si la donnée stockée est corrompue
- ☐ Je sais expliquer pourquoi on passe une *fonction* à `useState` dans ce hook
- ☐ Je sais justifier l'approche que j'ai choisie en B

AIDE

Piste

Deux moitiés : lire la valeur stockée au démarrage, et écrire à chaque changement. La lecture se fait une seule fois ; l'écriture à chaque modification.

`localStorage` ne sait stocker que du **texte**. Un tableau doit donc être converti dans les deux sens.

Indice

`useState` accepte une **fonction** comme valeur initiale : React ne l'appelle alors qu'au premier rendu. Sans ça, tu lirais le `localStorage` à chaque rendu pour jeter le résultat.

`JSON.stringify` et `JSON.parse` pour la conversion. Et un `try / catch` autour de la lecture : le `localStorage` peut être indisponible (navigation privée) ou contenir des données corrompues.

Squelette

```
export function useLocalStorage(cle, valeurInitiale) {
  const [valeur, setValeur] = useState(() => {
    // lire, convertir, gérer le cas "rien de stocké" et le cas d'erreur
  });

  useEffect(() => {
    // écrire
  }, [/* ... */]);

  return [valeur, setValeur];
}
```

LE MOMENT À SAVOURER

Tu viens d'ajouter la persistance à ton application en changeant **un seul mot** : `useState` est devenu `useLocalStorage`. C'est exactement ce que permettent les hooks personnalisés.

Le thème

Context API · Provider · hook d'accès

≈ 45 min

OBJECTIF

Partager une donnée dans tout l'arbre sans la passer de composant en composant.

Ce que tu dois faire

A — D'abord, à la main. Ajoute un thème clair / sombre en le passant **par props**. Le bouton de bascule doit être dans le `Header`, et la couleur des cartes doit dépendre du thème.

Compte le nombre de composants qui transportent la donnée sans s'en servir. Note ce chiffre.

B — Puis avec Context. Créer `src/contexts/ThemeContext.jsx` contenant :

- un composant `ThemeProvider` qui détient le state,
- un hook `useTheme()` pour y accéder, avec un message d'erreur clair si le Provider a été oublié.

C. Applique le thème en posant un attribut sur la balise `<html>`, et fais tout le reste en CSS avec des variables. React ne doit gérer qu'un seul mot.

D. Combine avec le palier 10 : le thème choisi survit au rechargement.

E. Teste ton garde-fou : retire temporairement le Provider et vérifie que le message d'erreur est explicite.

CRITÈRES DE RÉUSSITE

- ☐ Le bouton du `Header` change le thème de toute l'application
- ☐ Aucune prop `theme` ne traverse un composant intermédiaire
- ☐ Oublier le Provider produit un message d'erreur compréhensible
- ☐ Le thème est conservé après un rechargement
- ☐ Les composants importent `useTheme`, jamais le contexte directement

AIDE

Piste

Trois étapes : créer le contexte, fournir une valeur à un endroit haut de l'arbre, consommer cette valeur n'importe où en dessous.

Une image : Context, c'est le wifi de la maison. Tu ne tires pas un câble jusqu'à chaque appareil ; tu poses une box et tout ce qui est dans la maison peut s'y connecter.

Indice

`createContext`, `<MonContexte.Provider value={...}>`, et `useContext`. Dans `value`, mets à la fois la donnée et la fonction pour la changer.

Le garde-fou : dans ton hook d'accès, si `useContext` retourne la valeur par défaut, lève une erreur explicite. Trois lignes qui t'économiseront une demi-heure un jour.

Pour appliquer le thème : `document.documentElement.dataset.theme = ...` dans un effet, puis un sélecteur CSS `[data-theme="sombre"]` qui redéfinit tes variables.

Squelette

```
const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = /* ... */;
  useEffect(() => { /* appliquer au DOM */ }, [theme]);
  return (
    <ThemeContext.Provider value={{ /* ... */ }}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const contexte = useContext(ThemeContext);
  if (!contexte) throw new Error(/* ... */);
  return contexte;
}
```

LE GARDE-FOU À RETENIR

Context transporte, il ne range pas. Le state reste dans un `useState` ordinaire, à l'intérieur du Provider.

Et surtout : tout composant qui consomme un contexte se re-render quand sa valeur change. Ne mets **pas** tout ton état applicatif dans un contexte unique : c'est le moyen le plus efficace de rendre une application lente. Un contexte par sujet. Ta recherche et tes filtres, eux, restent où ils sont.

Le scroll infini

useRef · IntersectionObserver · événement global · cleanup

≈ 45 min

OBJECTIF

Afficher les pays par paquets, et charger la suite automatiquement quand l'utilisateur arrive en bas.

Ce que tu dois faire

A — La pagination. Un état « nombre d'éléments affichés », initialisé à 24. La grille n'en montre que ce nombre. Un bouton « Voir plus » en ajoute 24.

B — La sentinelle. Remplace le bouton par un élément vide placé *après* la grille, surveillé par un `IntersectionObserver`. Quand il entre à l'écran, la suite se charge toute seule.

C — Un événement global. Un bouton « ↑ Haut de page » qui n'apparaît qu'après 400 pixels de défilement. Il faut donc écouter le défilement de la fenêtre.

D — La remise à zéro. Quand la recherche ou le filtre changent, le compteur doit revenir à 24. Teste sans cette étape d'abord, pour voir le problème que ça pose.

E. Extrais B et C dans deux hooks personnalisés réutilisables, qui ne savent rien des pays.

CRITÈRES DE RÉUSSITE

- ☐ 24 pays au chargement, la suite arrive au défilement sans clic
- ☐ Le bouton « Haut de page » apparaît et disparaît correctement
- ☐ Faire une recherche remet le compteur à 24
- ☐ Tous les effets qui « ouvrent » quelque chose le referment
- ☐ Mes deux hooks fonctionneraient dans n'importe quel autre projet

AIDE

Piste

Ces deux fonctionnalités ont l'air différentes et c'est **le même exercice** : je m'accroche à quelque chose d'extérieur à React, donc j'ouvre dans un effet et je referme dans le nettoyage.

Pour B : tu as besoin de désigner un élément du DOM à surveiller. Quel hook permet de pointer vers un élément ?

Indice

Un piège classique : pour retirer un écouteur d'événement, il faut passer **exactement la même fonction** qu'à l'ajout. Deux fonctions fléchées écrites à l'identique sont deux fonctions différentes pour JavaScript. Nomme ta fonction.

Autre piège : dans le nettoyage d'un effet, `ref.current` peut déjà valoir `null`. Copie-le dans une variable locale au début de l'effet.

`new IntersectionObserver(callback)`, puis `.observe(element)`, et `.disconnect()` au nettoyage. La propriété à lire s'appelle `isIntersecting`.

Squelette

```
export function useInfiniteScroll(onVisible) {
  const sentinelle = useRef(null);

  useEffect(() => {
    const element = sentinelle.current; // copie locale
    if (!element) return;

    const observateur = new IntersectionObserver(/* ... */);
    observateur.observe(element);

    return () => /* ... */;
  }, [/* ... */]);

  return sentinelle;
}
```

UN PIÈGE QUI VA TE TOMBER DESSUS

Si tu passes une fonction fléchée créée directement dans le rendu à ton hook, l'observateur risque d'être détruit et recréé à chaque rendu. Regarde les dépendances de ton effet et demande-toi si elles changent à chaque fois. Il existe un hook fait pour stabiliser une fonction : cherche lequel.

Le mode Quiz

setInterval · cleanup · tableau de refs

≈ 60 min

OBJECTIF

Un mini-jeu chronométré, et la découverte d'un piège classique des timers.

Ce que tu dois faire

A. Une vue Quiz : un drapeau tiré au hasard, quatre propositions (le bon pays et trois autres au hasard), dans un ordre mélangé.

B — L'expérience à mener. Ajoute un chronomètre de 10 secondes qui descend. Écris-le d'abord de la façon la plus naturelle :

```
setInterval(() => setTemps(temps - 1), 1000);
```

Fais tourner. **Que se passe-t-il ?** Cherche pourquoi avant de corriger. Le terme technique est « closure périmée » (*stale closure*).

C. À zéro, la question est perdue et on passe à la suivante. Ajoute un score.

D. Un bouton Pause / Reprendre.

E. Navigation au clavier : les touches 1 à 4 sélectionnent une réponse et déplacent le focus sur le bouton correspondant.

F. Après avoir répondu, colorer la bonne réponse en vert, et la mauvaise en rouge si l'utilisateur s'est trompé.

CRITÈRES DE RÉUSSITE

- ☐ Le chronomètre descend correctement de 10 à 0
- ☐ Pause arrête vraiment le décompte, Reprendre le relance
- ☐ Quitter la vue Quiz arrête le timer — vérifie-le avec un `console.log`
- ☐ Les touches 1 à 4 fonctionnent
- ☐ Je sais expliquer ce qu'est une closure périmée
- ☐ Je sais dire quelle ligne de mon code arrête le timer au clic sur Pause

AIDE

Piste

Pour B : ton effet ne s'exécute qu'une fois. La fonction donnée à `setInterval` a été créée à ce moment-là. Que valait ta variable de temps à ce moment-là ? Est-ce qu'elle peut changer, pour cette fonction ?

Pour D : n'essaie pas de « démarrer » et « arrêter » le timer à la main. Fais l'inverse : décris ce qui doit *exister* quand le jeu est en cours, et laisse React créer et détruire le timer.

Pour E : tu ne peux pas appeler `useRef()` dans une boucle. Cherche pourquoi, puis cherche la solution.

Indice

La forme fonctionnelle d'un setter, `setX(valeurActuelle => ...)`, ne lit jamais la variable : il n'y a donc plus rien à périmer. Dans un timer, utilise-la toujours.

Pour le tableau de refs : une seule ref qui contient un tableau, remplie avec la forme fonction de l'attribut `ref`.

Pour Pause : mets un booléen dans les dépendances de ton effet, et sors de l'effet tout de suite s'il est faux.

Squelette

```
useEffect(() => {
  if (!enCours) return;          // rien à faire, donc pas de timer

  const id = setInterval(() => {
    setTemps(/* forme fonctionnelle */);
  }, 1000);

  return () => /* ... */;
}, [/* ... */]);

// pour les boutons
const boutonsRef = useRef([]);
<button ref={element => (boutonsRef.current[i] = element)} ... />
```

LE VRAI ENSEIGNEMENT DE CE PALIER

Quand ça marchera, relis ton code et cherche la ligne où tu as écrit « arrête le timer quand on clique sur Pause ».

Tu ne l'as pas écrite. Tu as décrit un état — « quand le jeu est en cours, il existe un intervalle » — et React s'est occupé du reste. C'est la même idée que `UI = f(state)` vue au tout début, appliquée à autre chose que l'affichage. Prends une minute là-dessus, c'est le cœur de React.

Les composants boîte

props.children · composition

≈ 35 min

OBJECTIF

Ouvrir le détail d'un pays dans une modale, et comprendre pourquoi un conteneur ne doit pas connaître son contenu.

Ce que tu dois faire

A — La mauvaise approche, volontairement. Fais une `Modale` avec des props `titre`, `texte`, `imageUrl`, `boutonLabel`. Puis demande-toi comment y ajouter un tableau de données. Puis deux boutons. Puis un lien.

Compte les props que tu devrais ajouter. Arrête-toi là.

B — La composition. Refais la `Modale` pour qu'elle ne connaisse **rien** de son contenu. Compare les deux appels côte à côte.

C. Cliquer sur une carte ouvre la modale avec le détail complet du pays.

D. La modale se ferme de trois façons : la croix, un clic sur le fond sombre, et la touche Échap.

E. Extrais un composant `Card` générique, et fais en sorte que `CountryCard` l'utilise. `Card` gère la boîte (bordure, ombre, arrondi) ; `CountryCard` ne s'occupe que du contenu.

CRITÈRES DE RÉUSSITE

- ☐ La modale ne connaît rien du contenu qu'on lui donne
- ☐ Échap la ferme, et l'écouteur est bien retiré ensuite
- ☐ Un clic sur le fond ferme, un clic sur le contenu ne ferme pas
- ☐ Cliquer sur l'étoile n'ouvre pas la modale
- ☐ `Card` est réutilisé par `CountryCard`
- ☐ Je sais expliquer pourquoi la composition vaut mieux qu'une longue liste de props

AIDE

Piste

Il existe une prop spéciale qui correspond à ce que tu écris *entre* les balises ouvrante et fermante d'un composant. Cherche son nom.

Pour D et E : quand tu cliques sur un élément, l'événement **remonte** vers ses parents. C'est ce qui fait que ton clic sur l'étoile déclenche aussi le clic de la carte. Il existe une méthode de l'objet événement pour arrêter cette remontée.

Indice

`children`. Et pour t'en convaincre, essaie ces deux écritures : `<Card><p>Salut</p></Card>` et `<Card children={<p>Salut</p>} />`. Ce n'est pas de la magie, juste un raccourci d'écriture.

Pour l'événement : `e.stopPropagation()` . Pour Échap : un écouteur `keydown` dans un effet, avec son nettoyage — tu sais déjà faire depuis le palier 12.

Pense aussi à l'accessibilité : une modale a un `role` et un nom accessible.

Squelette

```
function Modale({ children, onClose, titre }) {
  useEffect(() => { /* écouteur Échap + nettoyage */ }, [/* ... */]);

  return (
    <div className="modale-fond" onClick={/* ... */}>
      <div className="modale-contenu" onClick={/* ... */} role="dialog">
        <button onClick={onClose} aria-label="Fermer">x</button>
        {children}
      </div>
    </div>
  );
}
```

La performance

useMemo · React.memo · useCallback · **et surtout : mesurer**

≈ 40 min

OBJECTIF

Savoir que ces outils existent, ce qu'ils font — et savoir qu'il ne faut pas s'en servir par défaut.

À LIRE AVANT DE COMMENCER

Ce palier est un piège. Tu vas apprendre trois outils, et la consigne principale est : **ne les utilise pas par défaut**.

Un re-render React est très rapide : React compare le résultat au précédent et ne touche au DOM que s'il y a une différence. Un re-render inutile coûte quelques microsecondes — ce n'est pas un problème de performance. Ces trois outils, eux, coûtent de la mémoire, des comparaisons, et beaucoup de lisibilité.

Ce que tu dois faire — un exercice de mesure

Installe les **React DevTools** et ouvre l'onglet Profiler. Puis, à chaque étape, **note le chiffre** dans un tableau.

Étape	Modification	Temps de rendu
1	État actuel, sans rien — tape dans la recherche	... ms
2	Filtrage et tri enveloppés dans un <code>useMemo</code>	... ms
3	<code>CountryCard</code> enveloppé dans <code>React.memo</code>	... ms
4	Handlers stabilisés avec <code>useCallback</code>	... ms
5	Sans la pagination du palier 12 (250 cartes affichées)	... ms

Puis **conclus honnêtement**, par écrit :

- Quelle étape a apporté un gain réel, et de combien ?
- Y a-t-il une étape qui n'a rien apporté du tout ? Pourquoi, à ton avis ?
- Quelle est la modification qui a eu le plus d'effet sur la fluidité ?

Retire les optimisations qui n'ont rien apporté. C'est la partie la plus importante de l'exercice.

CRITÈRES DE RÉUSSITE

- ☐ Le tableau de mesures est rempli avec de vrais chiffres
- ☐ Ma conclusion écrite est honnête, même si elle contredit ce que j'attendais
- ☐ Les optimisations sans effet mesuré ont été retirées
- ☐ Je sais expliquer pourquoi `useCallback` tout seul ne sert à rien
- ☐ Je sais citer une optimisation qui n'utilise aucun de ces trois hooks

AIDE

Piste

Pour l'étape 2 : regarde les dépendances de ton `useMemo` . Est-ce que l'une d'elles change à chaque frappe ? Si oui, que mémorise-t-on exactement ?

Pour l'étape 3 : `React.memo` compare les props. Est-ce que toutes tes props sont identiques d'un rendu à l'autre ? Regarde bien celles qui sont des fonctions.

Indice

Une fonction fléchée écrite dans le rendu est une **nouvelle** fonction à chaque rendu. Donc une prop qui a changé. Donc un `React.memo` qui échoue systématiquement. C'est le même mécanisme que l'écouteur d'événement du palier 12.

D'où la règle : `useCallback` seul ne sert à rien. Il n'a d'intérêt que pour nourrir un `React.memo` ou une dépendance d'effet.

Attention aux dépendances

Tout ce qu'utilise ton `useMemo` doit figurer dans son tableau de dépendances. Une dépendance oubliée affiche des données périmées à l'écran — c'est un bug bien plus grave qu'une lenteur.

LA RÈGLE À EMPORTER

N'optimise pas avant d'avoir constaté un ralentissement, mesuré la cause, et vérifié que ta correction améliore les choses. Si le chiffre n'a pas bougé, retire ta modification.

Et retiens ceci : la plupart des problèmes de performance en React ne se règlent pas avec les hooks de performance de React. Ils se règlent en affichant moins de choses.

Le style

CSS Modules · Tailwind

≈ 35 min

OBJECTIF

Sortir du CSS global et choisir une approche en connaissance de cause.

Ce que tu dois faire

A. Convertis `Card` en **CSS Modules**. Puis inspecte le nom de classe généré dans le navigateur : qu' observes-tu ? À quoi ça sert ?

B. Convertis `Modal` également, avec une classe conditionnelle au passage.

C. Refais le composant `Badge` (« Grand pays » / « Petit pays ») en **Tailwind**.

D — La comparaison. Remplis ce tableau avec tes propres observations, puis écris trois lignes sur ce que tu choisirais pour ton prochain projet et pourquoi.

	CSS Modules	Tailwind
Nombre de fichiers		
Lignes écrites		
Lisibilité du JSX		
Ajouter une variante		

CRITÈRES DE RÉUSSITE

- ☐ Deux composants au moins utilisent les CSS Modules
- ☐ Un composant utilise Tailwind
- ☐ Le tableau de comparaison est rempli
- ☐ J'ai un avis argumenté, fondé sur ce que j'ai fait et pas sur ce que j'ai lu

AIDE

Piste

Le nom du fichier compte : c'est une convention précise qui déclenche le traitement par le bundler. Cherche laquelle.

Indice

`MonComposant.module.css`, puis `import styles from './MonComposant.module.css'`, et `className={styles.maClasse}`.

Pour combiner plusieurs classes, un template literal suffit. Il existe aussi une petite librairie très répandue pour ça : cherche `clsx`.

À vérifier

La procédure d'installation de Tailwind a changé entre ses versions récentes. Consulte la documentation officielle du jour plutôt qu'un tutoriel trouvé au hasard.

LA SEULE VRAIE RÈGLE

Une seule approche par projet. Les deux sont massivement utilisées en production ; c'est le mélange qui rend un projet ingérable.

BONUS

Aller plus loin

React Router · TanStack Query · Testing Library

Ces trois chantiers dépassent le programme des deux modules. À faire en autonomie, dans cet ordre.

Bonus 1 — React Router

Transforme ton application en vraie application multi-pages :

- `/` — la grille,
- `/pays/:id` — le détail sur une page à part entière plutôt qu'en modale,
- `/quiz` — le mode quiz,
- `/favoris` — la liste des favoris,
- un layout avec une navigation qui reste affichée,
- une vraie page 404.

Le vrai exercice : mets le terme de recherche dans l'URL plutôt que dans un state. Tu pourras alors envoyer un lien vers une recherche précise, et un rafraîchissement conservera tout.

L'URL est un état — le seul qui soit partageable et présent dans l'historique du navigateur.

Bonus 2 — TanStack Query

Remplace ton hook `useCountries` par un `useQuery`. Avant de commencer, réponds à ces questions sur ta version actuelle :

1. Si deux composants demandent les mêmes données, combien de requêtes partent ?
2. Si l'utilisateur quitte la page et revient, que voit-il ?
3. Si la connexion coupe une seconde, que se passe-t-il ?

Puis refais le test après la migration. La différence est le sujet de l'exercice.

Les données du serveur ne sont pas ton état : tu en as une copie qui peut être périmée. `useState` sert à ce que tu possèdes ; TanStack Query gère ce que tu empruntes.

Bonus 3 — Testing Library

Par ordre de rentabilité :

1. **Le réduire du palier 9.** Trois lignes par test, aucun mock. Teste aussi qu'il ne modifie pas l'état d'origine.
2. **La recherche.** Taper un terme, vérifier que la liste se filtre.
3. **Le cas d'erreur** de l'API.

Teste ce que l'utilisateur voit et fait, jamais comment c'est écrit à l'intérieur. Un test qui connaît tes noms de variables casse au premier renommage, alors que l'application marche parfaitement.

Références

A — L'arborescence visée en fin de projet

```
src/
  components/
    Header.jsx
    CountryCard.jsx
    Card.jsx           Card.module.css
    Modale.jsx         Modale.module.css
    Filtres.jsx
    Grille.jsx
    Quiz.jsx
    AllerAuPays.jsx
  hooks/
    useCountries.js
    useLocalStorage.js
    useInfiniteScroll.js
    useScrollPosition.js
  contexts/
    ThemeContext.jsx
  reducers/
    listeReducer.js
  data/
    countries.js
  App.jsx
  main.jsx
  index.css
```

Chaque dossier correspond à une notion du cours. Et surtout : aucun fichier de plus de 150 lignes. C'est le vrai résultat du projet.

B — Le glossaire

Terme	Ce que ça veut dire
Rendu	React appelle ta fonction composant et regarde ce qu'elle retourne
Re-render	Il la rappelle parce que quelque chose a changé
Monter / démonter	Le composant apparaît à l'écran / en disparaît
State	Une valeur qui, quand elle change, déclenche un nouveau rendu
Props	Les paramètres qu'un composant reçoit de son parent
Hook	Une fonction commençant par <code>use</code> qui donne accès aux capacités de React
Effet de bord	Ce qui sort du calcul : appel réseau, timer, écouteur d'événement
Cleanup	Le <code>return</code> d'un <code>useEffect</code> : ce qu'on défait quand le composant part
Dépendances	Le tableau d'un <code>useEffect</code> : les valeurs dont il a besoin
Valeur dérivée	Une valeur calculée à partir d'un état, qu'on ne stocke pas

Immutabilité	Ne jamais modifier un objet ou un tableau : en créer une copie modifiée
Lifting state up	Remonter un état dans le parent commun de ceux qui en ont besoin
Prop drilling	Passer une donnée à travers des composants qui ne s'en servent pas
Provider	Un composant qui rend une donnée disponible à tout ce qu'il contient
Closure périmée	Une fonction qui a gardé en mémoire une ancienne valeur

C — Les erreurs que tu vas rencontrer

Ce tableau ne donne pas la solution, seulement la direction où chercher.

Le message ou le symptôme	Où chercher
Cannot read property 'map' of null	Avec quoi as-tu initialisé ton state ?
Requêtes à l'infini, onglet qui gèle	Le tableau de dépendances de ton effet
Bloqué sur « Chargement... »	Où as-tu mis la fin du chargement ?
Une page d'erreur affichée comme des données	As-tu vérifié le statut de la réponse ?
Rien ne se met à jour après un ajout au tableau	As-tu modifié le tableau, ou créé un nouveau ?
Le compteur reste bloqué à une valeur	Ta fonction de timer lit-elle une variable extérieure ?
Each child should have a unique key	Ton <code>.map()</code>
Rendered more hooks than during the previous render	Un hook est-il dans un <code>if</code> ou après un <code>return</code> ?
Le formulaire recharge la page	Comportement natif du HTML — que faut-il appeler ?
Deux <code>set...</code> d'affilée ne font qu'un seul changement	La valeur est figée pour ce rendu — quelle forme utiliser ?
Un effet qui s'exécute deux fois en développement	C'est voulu. Cherche « StrictMode » — et vérifie ton nettoyage.
401 ou 403 depuis l'API	Ta clé, son en-tête, ou ton quota mensuel

D — Auto-évaluation de fin de projet

Coche honnêtement. Ce qui n'est pas coché indique ce qu'il faut retravailler.

JE SAIS EXPLIQUER, SANS RELIRE MES NOTES

- ☐ Pourquoi une variable normale ne suffit pas et pourquoi `useState` existe
- ☐ Pourquoi mon composant s'affiche deux fois pendant un chargement de données
- ☐ Ce qu'est une valeur dérivée, et pourquoi il ne faut pas la stocker
- ☐ Pourquoi un hook ne peut pas être appelé dans une condition
- ☐ À quoi sert vraiment la `key` dans une liste
- ☐ Pourquoi il ne faut jamais modifier un state directement

- ☐ Quand passer de `useState` à `useReducer`
- ☐ Ce que `Context` transporte, et ce qu'il ne fait pas
- ☐ Pourquoi un effet qui ouvre quelque chose doit le refermer
- ☐ Ce qu'est une closure périmée et comment l'éviter
- ☐ Pourquoi `useCallback` seul ne sert à rien
- ☐ Pourquoi on mesure avant d'optimiser

MON CODE RESPECTE

- ☐ Aucun fichier de plus de 150 lignes
- ☐ Aucun état redondant : les valeurs dérivées sont calculées
- ☐ Tous les effets qui ouvrent quelque chose ont un nettoyage
- ☐ Les trois états de l'interface sont gérés partout où il y a des données
- ☐ L'immutabilité est respectée : aucun `push` , aucune modification directe
- ☐ Les images ont un `alt` , les boutons-icônes ont un `aria-label`
- ☐ Aucun avertissement dans la console

ET APRÈS ?

La meilleure façon de continuer, c'est de construire. Prends une idée que tu as envie de voir exister et fais-la. Tu apprendras davantage en trois jours de projet raté qu'en trois semaines de tutoriels réussis.